

SABUESO DE LICITACIONES

Documentación Técnica Completa

Pipeline de Monitorización de Licitaciones Públicas Españolas

Versión 1.0 · Junio 2026

Este documento describe en detalle la arquitectura, los componentes técnicos y el código fuente del proyecto Sabueso de Licitaciones: un sistema automatizado de monitorización de la Plataforma de Contratación del Sector Público (PCSP) que descarga licitaciones, extrae documentos, analiza su relevancia con IA y notifica a los clientes interesados.

Índice de Contenidos

1. ¿Qué es Sabueso de Licitaciones? — Propósito, problema que resuelve y modelo de negocio
2. Conceptos fundamentales — PCSP, feed ATOM, UBL, licitaciones, CPV y pipeline
3. Arquitectura del sistema — Estructura de carpetas, capas y flujo de datos
4. La máquina de estados — Estados del pipeline y transiciones
5. Módulo por módulo — Explicación detallada de cada archivo Python
6. La base de datos — Tablas, relaciones y modelo multicliente
7. Configuración — Variables de entorno y config.py
8. Dependencias — Librerías usadas y por qué cada una
9. Cómo ejecutar el proyecto — Comandos CLI y ejemplos
10. Estado actual y roadmap — Qué funciona, qué falta, próximos pasos

1. ¿Qué es Sabueso de Licitaciones?

Sabueso de Licitaciones es un sistema automatizado de vigilancia de contratos públicos en España. Su función es monitorizar continuamente la Plataforma de Contratación del Sector Público (PCSP), que es la web oficial del Gobierno donde todas las administraciones están obligadas a publicar sus licitaciones.

El problema que resuelve

Cada día se publican cientos de licitaciones públicas en España: ayuntamientos que buscan proveedores de software, hospitales que necesitan sistemas de gestión, universidades que contratan servicios IT, etc. Una empresa que quiera optar a estos contratos tendría que revisar manualmente la web de la PCSP cada día, lo cual es inviable en la práctica.

Cómo lo resuelve Sabueso

- **Descarga automática:** Cada vez que se ejecuta, descarga las últimas licitaciones publicadas en la PCSP a través de su feed ATOM oficial.
- **Descarga de documentos:** Para cada licitación, descarga automáticamente el PDF del pliego de condiciones (el documento donde se detalla qué se pide y cómo presentarse).
- **Extracción de texto:** Lee el PDF y extrae su texto. Si el PDF es una imagen escaneada, usa IA para leerlo (OCR).
- **Análisis de relevancia:** Compara el texto del pliego contra el perfil del cliente usando inteligencia artificial (Gemini). Le asigna una puntuación de 0 a 100.
- **Notificación:** Si la licitación supera el umbral de puntuación, avisa al cliente (Step 5, pendiente de implementar).

Modelo de negocio

Sabueso está diseñado como un SaaS (Software as a Service) B2B. Esto significa que el sistema es multicitiente: varias empresas pueden usar el mismo Sabueso, cada una con sus propios criterios de búsqueda (perfiles) y recibiendo solo las licitaciones que les interesan. Cada cliente paga una suscripción mensual por el servicio.

2. Conceptos Fundamentales

La PCSP (Plataforma de Contratación del Sector Público)

Es el portal oficial del Gobierno de España donde todas las administraciones públicas (ayuntamientos, diputaciones, comunidades autónomas, ministerios, universidades públicas, hospitales públicos, etc.) están legalmente obligadas a publicar sus licitaciones. URL: contrataciondelsectorpublico.gob.es

El Feed ATOM

Un feed ATOM es un archivo XML (texto estructurado) que se actualiza automáticamente con los últimos datos. Es similar a los feeds RSS de noticias, pero para licitaciones. La PCSP publica su feed en una URL pública que cualquiera puede consultar.

El feed que usamos contiene hasta 350-500 licitaciones y se actualiza diariamente. Cada entrada del feed tiene campos estructurados con los metadatos de la licitación: título, órgano contratante, presupuesto, fechas, y la URL del documento PDF.

UBL (Universal Business Language)

El feed de la PCSP usa el estándar europeo UBL para estructurar los datos. Esto significa que los metadatos ricos (presupuesto, localización, URL del PDF) están disponibles como campos directos del feed, sin necesidad de navegar páginas web. Los campos UBL que usamos en el proyecto tienen nombres como `cbc_uri` (URL del PDF), `cac_budgetamount` (presupuesto), `cbc_cityname` (ciudad), etc.

El CPV (Vocabulario Común de Contratos)

El CPV es un sistema europeo de clasificación de contratos públicos mediante códigos numéricos. Cada licitación lleva uno o varios códigos CPV que indican de qué tipo es el contrato. Por ejemplo: los códigos que empiezan por 72 son servicios informáticos, los que empiezan por 48 son licencias de software, los que empiezan por 45 son obras de construcción. Sabueso usa estos códigos como primer filtro rápido.

El concepto de Pipeline

Un pipeline (tubería en inglés) es una secuencia de pasos donde la salida de uno es la entrada del siguiente. En Sabueso, cada licitación pasa por 5 steps (pasos) en orden. Si falla en algún paso, queda en estado ERROR y no avanza. Esto permite reintentar solo los pasos que fallaron sin rehacer los que ya funcionaron.

3. Arquitectura del Sistema

Estructura de carpetas

```
sabueso_licitaciones/
├──
├── src/                                ← Todo el código fuente
│   ├── __init__.py                    ← Marca la carpeta como paquete Python
│   ├── logger.py                      ← Sistema de logs centralizado
│   ├── models.py                     ← Estructura de datos (BD y transferencia)
│   ├── db_manager.py                 ← Capa de acceso a base de datos
│   ├── scraper_atom.py               ← Step 1: descarga e ingesta del feed ATOM
│   ├── downloader.py                 ← Step 2: descarga de PDFs
│   ├── extractor.py                  ← Step 3: extracción de texto de PDFs
│   ├── analizador.py                 ← Step 4: motor de relevancia (filtro + IA)
│   ├── ia_client.py                  ← Capa de abstracción para llamadas a Gemini
│   └── utils.py                      ← Funciones auxiliares puras
├──
├── scripts/
│   └── crear_cliente_prueba.py        ← Crea un cliente de prueba en BD
├──
├── tests/
│   └── test_utils.py                  ← Tests unitarios de utils.py
├──
├── alembic/                          ← Migraciones de base de datos
│   └── versions/                     ← Historial de cambios en el esquema BD
├──
├── data/pdfs/                        ← PDFs descargados (generado automáticamente)
├── database/                         ← sabueso.db - base de datos SQLite
├── logs/                             ← sabueso.log - logs del sistema
├──
├── config.py                         ← Fuente única de configuración
├── main.py                           ← Punto de entrada CLI (orquestador)
├── requirements.txt                   ← Dependencias Python
└── .env                              ← Variables de entorno (API keys, etc.)
```

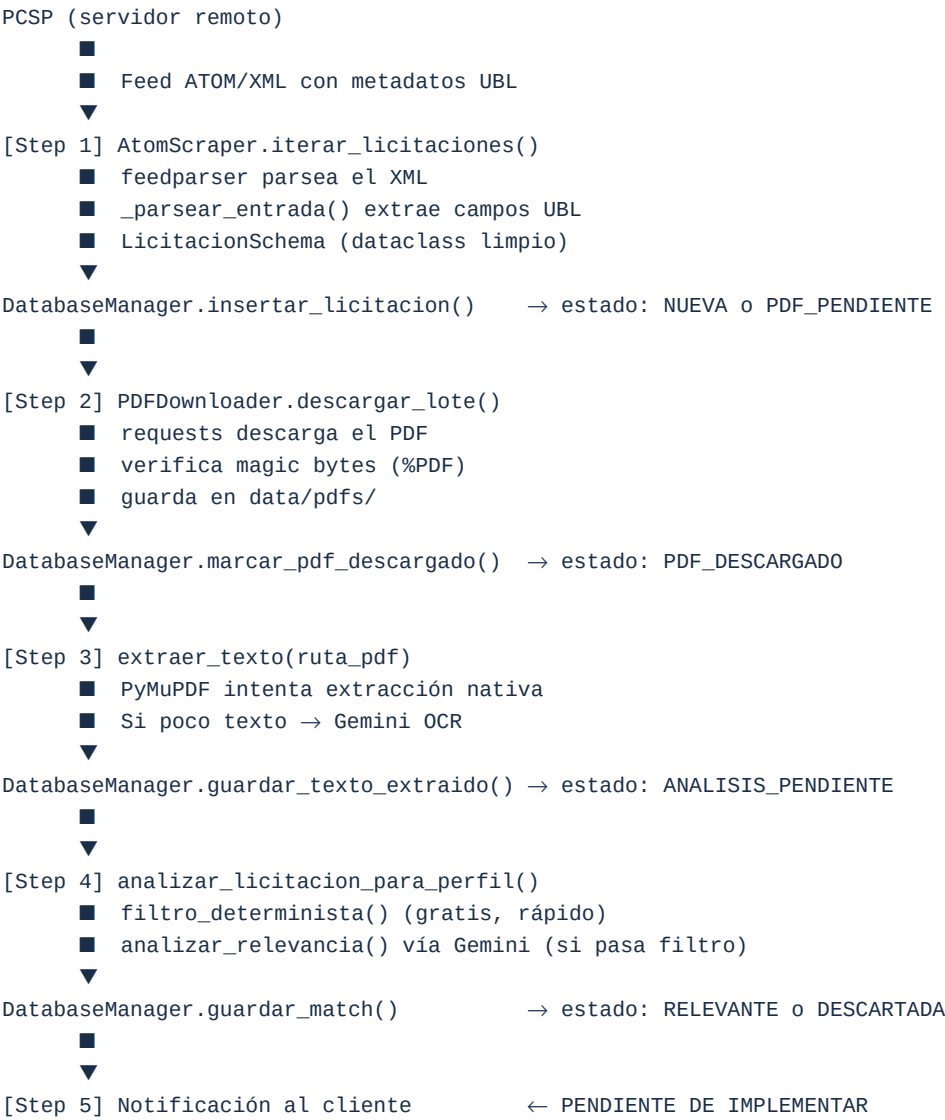
Principio de responsabilidad única

Cada módulo del proyecto tiene una única responsabilidad claramente definida y no hace lo que no le corresponde. Esto es importante porque hace el código más fácil de entender, depurar y modificar. La siguiente tabla resume qué hace y qué NO hace cada módulo:

Módulo	Sí hace	NO hace
scraper_atom.py	Descarga y parsea el feed ATOM	No escribe en BD, no descarga PDFs
downloader.py	Descarga PDFs a disco	No lee su contenido, no escribe en BD
extractor.py	Extrae texto de PDFs	No analiza relevancia, no guarda en BD
analizador.py	Decide si una licitación es relevante	No hace HTTP, no toca ficheros
ia_client.py	Habla con la API de Gemini	No sabe nada del negocio
db_manager.py	Lee y escribe en la BD	No hace HTTP, no parsea HTML

utils.py	Transforma datos (puro)	No hace HTTP, no toca BD ni disco
main.py	Orquesta los módulos en orden	No contiene lógica de negocio
config.py	Define toda la configuración	No importa ningún módulo del proyecto

Flujo de datos de principio a fin



4. La Máquina de Estados

Cada licitación en la base de datos tiene un campo estado_proceso que indica en qué punto del pipeline se encuentra. Esto es lo que permite al sistema ser resiliente: si el proceso se interrumpe a mitad, la próxima ejecución sabe exactamente desde dónde retomar sin repetir trabajo.

Estado	Icono	Descripción	Siguiente paso
NUEVA	■	Recién ingestada del feed. Sin PDF.	Descargar PDF
PDF_PENDIENTE	■	Tiene URL de PDF pero sin descargar.	Descargar PDF
PDF_DESCARGADO	■	PDF guardado en data/pdfs/.	Extraer texto
ANALISIS_PENDIENTE	■	Texto extraído. Esperando análisis IA.	Analizar con Gemini
ANALIZADA	■	IA la procesó (estado intermedio).	Clasificar resultado
RELEVANTE	■	IA: puntuación >= 60. Interesa al cliente.	Notificar
DESCARTADA	■■	IA: puntuación < 60. No relevante.	Fin
NOTIFICADA	■	Cliente informado por email/WhatsApp.	Fin
ERROR	■	Falló en algún paso del pipeline.	Reintentar manualmente

Nota: El umbral de relevancia está configurado en config.py como SCORE_RELEVANCIA_MINIMO = 60. Cualquier licitación con score >= 60 se considera RELEVANTE. Este valor se puede ajustar por cliente en el futuro.

5. Módulo por Módulo

Esta sección explica en detalle cada archivo Python del proyecto: qué hace, por qué está diseñado así, y las partes más importantes de su código.

5.1 config.py — Fuente única de configuración

Este archivo es el primero que se ejecuta al importar el proyecto. Contiene TODA la configuración: rutas de carpetas, URLs del feed, parámetros de red, configuración de logs, y las claves de API.

Por qué es importante tener toda la configuración en un solo sitio: si mañana quieres cambiar el timeout de red de 30 a 60 segundos, lo cambias en un solo lugar y afecta a todo el proyecto. Sin este patrón, tendrías que buscar el número 30 disperso por todo el código.

```
# Ejemplo de config.py simplificado:

# La ruta BASE_DIR apunta a la carpeta raíz del proyecto.
# Path(__file__).parent.resolve() significa:
# "la carpeta que contiene este mismo archivo (config.py)"
BASE_DIR = Path(__file__).parent.resolve()

# Subcarpetas que se crean automáticamente si no existen
DATA_DIR = BASE_DIR / "data" / "pdfs" # Aquí van los PDFs
DATABASE_DIR = BASE_DIR / "database" # Aquí va la BD SQLite

# load_dotenv() lee el archivo .env y carga sus variables como
# variables de entorno del sistema operativo
load_dotenv(BASE_DIR / ".env")

# os.getenv() lee una variable de entorno.
# El segundo argumento es el valor por defecto si no existe.
GEMINI_API_KEY = os.getenv("GEMINI_API_KEY", "")
GEMINI_MODEL = os.getenv("GEMINI_MODEL", "gemini-2.5-flash-lite-preview-06-17")

# La clase EstadoLicitacion agrupa las constantes de estado
# como atributos de clase. Es una forma ordenada de evitar
# escribir strings como "NUEVA" sueltos por el código.
class EstadoLicitacion:
    NUEVA = "NUEVA"
    PDF_PENDIENTE = "PDF_PENDIENTE"
    PDF_DESCARGADO = "PDF_DESCARGADO"
    ANALISIS_PENDIENTE = "ANALISIS_PENDIENTE"
    RELEVANTE = "RELEVANTE"
    DESCARTADA = "DESCARTADA"
    ERROR = "ERROR"
```

5.2 logger.py — Sistema de logs centralizado

Un log es un registro de lo que el programa va haciendo mientras se ejecuta, con fecha y hora. Son imprescindibles para saber qué pasó cuando algo falla sin estar mirando la pantalla.

Este módulo define una fábrica de loggers: una función `get_logger()` que cualquier otro módulo puede llamar para obtener su propio logger, ya configurado con el formato correcto y sin duplicar handlers.

```
# Cómo usa cualquier módulo el sistema de logs:
from src.logger import get_logger
logger = get_logger(__name__) # __name__ es el nombre del módulo actual

# Niveles de log (de menor a mayor gravedad):
logger.debug("Mensaje de depuración - solo se ve en modo DEBUG")
logger.info("Información normal - progreso del programa")
logger.warning("Aviso - algo raro pero no es un error")
logger.error("Error - algo falló pero el programa continúa")
logger.critical("Error crítico - el programa no puede continuar")

# El sistema escribe en DOS sitios a la vez:
# 1. La consola (stdout) - ves el progreso en tiempo real
# 2. logs/sabueso.log - fichero con rotación automática (máx 10MB)
```

5.3 models.py — Estructura de datos

Este módulo define todas las estructuras de datos del proyecto. Hay dos tipos de estructuras: los schemas de transferencia (dataclasses de Python puro) y los modelos ORM (clases que SQLAlchemy mapea a tablas de la base de datos).

¿Qué es un ORM?

ORM significa Object-Relational Mapper. Es una librería que te permite trabajar con la base de datos usando objetos Python en vez de escribir SQL directamente. En lugar de escribir INSERT INTO licitaciones (...) VALUES (...), simplemente creas un objeto Licitacion() y lo añades a la sesión.

```
# LicitacionSchema: dataclass de Python puro.
# Viaja desde el scraper hasta el db_manager.
# No depende de SQLAlchemy – es solo un contenedor de datos.
@dataclass
class LicitacionSchema:
    id: str # ID único de la licitación
    titulo: str # Título del contrato
    presupuesto_base: Optional[float] = None # En euros
    url_pdf_directo: Optional[str] = None # URL del PDF del pliego
    # ... más campos

# Licitacion: modelo ORM que SQLAlchemy mapea a la tabla 'licitaciones'.
# Cada atributo Mapped[] corresponde a una columna en la BD.
class Licitacion(Base):
    __tablename__ = "licitaciones"

    id: Mapped[str] = mapped_column(String(512), primary_key=True)
    titulo: Mapped[str] = mapped_column(Text, nullable=False)
    estado_proceso: Mapped[str] = mapped_column(String(30), default="NUEVA")
    texto_extraido: Mapped[Optional[str]] = mapped_column(Text)
    # ... más columnas
```

El modelo multicliente

Además de las licitaciones, models.py define las tablas del modelo de negocio: Cliente (una empresa suscrita al servicio), PerfilInteres (los criterios de búsqueda de ese cliente: palabras clave, CPV, presupuesto mínimo/máximo, provincias y descripción para la IA), y MatchLicitacion (el resultado del análisis de una licitación para un perfil concreto).

```

# Relaciones entre tablas:
# Un Cliente tiene MUCHOS PerfilInteres
# Un PerfilInteres tiene MUCHOS MatchLicitacion
# Una Licitacion tiene MUCHOS MatchLicitacion (uno por perfil analizado)

# La tabla MatchLicitacion es el corazón del sistema:
class MatchLicitacion(Base):
    __tablename__ = "matches"
    licitacion_id: Mapped[str]      # qué licitación
    perfil_id: Mapped[int]          # para qué perfil
    paso_filtro_a: Mapped[bool]    # pasó el filtro determinista?
    score_ia: Mapped[Optional[int]] # puntuación de Gemini (0-100)
    razon_ia: Mapped[Optional[str]] # por qué ese score
    notificado: Mapped[bool]       # ya se avisó al cliente?

# El índice unique en (licitacion_id, perfil_id) garantiza que
# una licitación solo se analiza UNA VEZ por perfil,
# aunque el pipeline se ejecute 100 veces.

```

5.4 scraper_atom.py — Step 1: Ingesta del feed ATOM

Este módulo es el punto de entrada del pipeline. Descarga el feed ATOM de la PCSP y convierte cada entrada en un LicitacionSchema listo para guardar en la BD.

Estrategia de descarga en 3 capas

El feed de la PCSP a veces devuelve XML malformado (esto ocurrió en las primeras ejecuciones de marzo 2026, como se ve en los logs). Para ser resiliente, el scraper tiene 3 estrategias en cascada: primero intenta feedparser nativo, luego repara el XML con lxml, y como último recurso usa BeautifulSoup.

```

# La clase AtomScraper encapsula toda la lógica de ingesta.
# Es 'stateless': no guarda estado entre llamadas.

class AtomScraper:
    def iterar_licitaciones(self, limite=20):
        """
        Generador que produce un LicitacionSchema por cada entrada.
        Un generador es una función que devuelve valores uno a uno
        con 'yield' en lugar de devolver todos a la vez con 'return'.
        Esto es eficiente en memoria: no carga las 350 licitaciones
        a la vez, sino que las procesa de una en una.
        """
        feed = self._descargar_feed()      # descarga con 3 estrategias
        for entry in feed.entries[:limite]:
            schema = self._parsear_entrada(entry) # convierte a dataclass
            if schema:
                yield schema                # devuelve de una en una

    def _parsear_entrada(self, entry):
        """
        Convierte una entrada UBL del feed en un LicitacionSchema.
        Jerarquía de extracción:
        1. Campos UBL directos (cbc_uri, cac_budgetamount...) - más fiables
        2. Summary (texto plano separado por ';') - como fallback
        3. Campos ATOM estándar (title, link) - siempre presentes
        """
        url_pdf = entry.get("cbc_uri")      # URL directa del PDF

```

```

nombre_pdf = entry.get("cbc_filename") # nombre descriptivo
presupuesto = parsear_budget_amount(    # extrae el importe base
    entry.get("cac_budgetamount")
)
# ... más campos

```

Backoff exponencial con tenacity

El decorador `@retry` de la librería `tenacity` hace que si el servidor de la PCSP falla, el sistema espere y lo reintente automáticamente. El 'backoff exponencial' significa que espera 2 segundos la primera vez, 4 la segunda, 8 la tercera... Esto evita sobrecargar el servidor cuando hay problemas.

5.5 downloader.py — Step 2: Descarga de PDFs

Este módulo descarga los PDFs de las licitaciones. La URL del PDF viene directamente del campo `cbc_uri` del feed UBL, por lo que no es necesario navegar páginas web con Playwright (una solución anterior más compleja).

```

# Verificación de magic bytes: asegura que lo descargado ES un PDF.
# Un PDF siempre empieza con los bytes '%PDF' (4 bytes).
# Sin esta verificación, podríamos guardar páginas de error HTML
# como si fueran PDFs, lo que rompería el Step 3 silenciosamente.

_PDF_MAGIC = b"%PDF"

def _es_pdf_valido(self, contenido: bytes) -> bool:
    return contenido[:4] == _PDF_MAGIC

# Si no pasa la verificación, el módulo registra un warning
# con el inicio del contenido para ayudar a diagnosticar el problema.

# Idempotencia: si el PDF ya existe en disco con tamaño > 0,
# no se re-descarga. Esto permite reintentar sin repetir trabajo.
if ruta_destino.exists() and ruta_destino.stat().st_size > 0:
    logger.debug("PDF ya existe, omitiendo descarga: %s", nombre_fichero)
    return ruta_destino

```

5.6 extractor.py — Step 3: Extracción de texto

Este módulo lee los PDFs descargados y extrae su texto. Maneja tres casos distintos: PDFs digitales (generados por ordenador), PDFs escaneados (imágenes) y archivos DOCX (que técnicamente son ZIPs con XML dentro).

```

# Estrategia en dos fases para extracción de texto:

def extraer_texto(ruta_fichero: Path) -> Optional[str]:

    # FASE 1: PyMuPDF (librería Python, gratis, milisegundos)
    # Si el PDF es digital (generado por Word, etc.), tiene texto
    # embebido que PyMuPDF puede extraer directamente.
    texto = _extraer_texto_pymupdf(ruta_fichero)

    if len(texto.strip()) >= 100: # config.TEXTO_MINIMO_CARACTERES
        return texto # PDF digital OK, terminamos aquí

```

```

# FASE 2: Gemini OCR (solo si PyMuPDF no encontró texto suficiente)
# Algunos órganos pequeños escanean sus documentos en papel.
# En ese caso, el PDF es literalmente una imagen y PyMuPDF
# no puede extraer texto. Gemini puede leer imágenes (OCR).
return extraer_texto_ocr(ruta_fichero) # llama a ia_client.py

# Extracción de DOCX:
# Un archivo .docx es un ZIP que contiene XML.
# El texto está en word/document.xml dentro de etiquetas <w:t>.
def _extraer_texto_docx(ruta_zip: Path) -> Optional[str]:
    with zipfile.ZipFile(str(ruta_zip), "r") as archivo_zip:
        xml_bytes = archivo_zip.read("word/document.xml")
        fragmentos = re.findall(r"<w:t[^>]*>([<]*)</w:t>", xml_bytes)
        return " ".join(fragmentos)

```

5.7 ia_client.py — Capa de abstracción para IA

Este módulo es la única puerta de comunicación con la IA (Gemini). Todo el proyecto pasa por aquí para hablar con la API. Si mañana se cambia de Gemini a otro proveedor, solo hay que modificar este archivo.

Función analizar_relevancia()

Le pasa a Gemini el texto del pliego y la descripción del perfil del cliente. Le pide que responda ÚNICAMENTE con un JSON con un score (0-100) y una razón. Luego parsea ese JSON y devuelve el resultado.

```

# El prompt que se envía a Gemini (resumido):
prompt = f"""
Eres un asistente especializado en licitaciones públicas españolas.
Analiza si la siguiente licitación es relevante para este perfil:

PERFIL: {descripcion_perfil}

LICITACIÓN:
- Título: {titulo}
- Presupuesto: {presupuesto_str}
- Texto: {texto_truncado} ← primeros 4000 caracteres

Responde ÚNICAMENTE con JSON:
{{"score": <0-100>, "razon": "<explicación breve>"}}
"""

# Por qué 4000 caracteres: un pliego puede tener 100.000 caracteres.
# Mandar todo sería caro y lento. Con 4000 chars hay suficiente
# contexto para que Gemini evalúe la relevancia correctamente.

# Por qué pedir JSON: porque es fácil de parsear con json.loads().
# Si Gemini respondiera en texto libre, sería muy difícil extraer
# el score de forma fiable.

```

5.8 analizador.py — Step 4: Motor de relevancia

Este módulo implementa el análisis de relevancia en dos fases. La Fase A es determinista y gratuita (no llama a la IA). La Fase B usa Gemini, pero solo para las licitaciones que pasaron la Fase A. Esto minimiza el coste de la IA.

```

# FASE A – Filtro determinista (gratis, milisegundos)
# Comprueba criterios objetivos configurados en el perfil del cliente.

def filtro_determinista(licitacion, perfil) -> tuple[bool, str]:

    # Filtro por CPV: si el perfil tiene prefijos CPV configurados,
    # la licitación debe tener un CPV que empiece por alguno de ellos.
    # Ejemplo: perfil tiene ["72", "48"], licitación tiene CPV "72100000"
    # → coincide con "72" → pasa el filtro
    cpv_prefijos = perfil.get_cpv_prefijos_lista()
    if cpv_prefijos and licitacion.cpv_codigo:
        coincide = any(
            licitacion.cpv_codigo.startswith(p) for p in cpv_prefijos
        )
        if not coincide:
            return False, f"CPV {licitacion.cpv_codigo} no coincide"

    # Filtro por presupuesto: rango mínimo/máximo configurado en el perfil
    if perfil.presupuesto_min and licitacion.presupuesto_base:
        if licitacion.presupuesto_base < perfil.presupuesto_min:
            return False, "Presupuesto por debajo del mínimo"

    # Filtro por palabras clave: busca en título + CPV + primeros 1000
    # chars del texto extraído
    palabras = perfil.get_palabras_clave_lista()
    if palabras:
        texto = (licitacion.titulo + " " + (licitacion.texto_extraido or "")[:1000]).lower()
        if not any(kw in texto for kw in palabras):
            return False, "Ninguna palabra clave encontrada"

    return True, "OK" # pasó todos los filtros

# FASE B – Solo si pasa la Fase A
# Si el perfil no tiene descripcion_ia, score por defecto = 70
# Si tiene descripcion_ia, llama a Gemini

```

5.9 db_manager.py — Capa de acceso a base de datos

Este módulo es la única puerta de entrada a la base de datos. Ningún otro módulo escribe o lee de la BD directamente — todo pasa por DatabaseManager. Esto centraliza el manejo de sesiones, commits y errores de BD.

```

# El context manager session() gestiona la sesión automáticamente.
# Al salir del bloque 'with', hace commit (guarda) o rollback (deshace).

with db.session() as session:
    db.insertar_licitacion(session, schema)
# <- aquí hace commit automático si no hubo excepción

# insertar_licitacion() es idempotente:
# si la licitación ya existe (mismo ID), la actualiza en vez de duplicar.
# Esto permite ejecutar el pipeline varias veces sin duplicados.

def insertar_licitacion(self, session, schema: LicitacionSchema):
    existente = session.get(Licitacion, schema.id)
    if existente:
        # Ya existe → actualizar solo campos que hayan cambiado

```

```

    existente.fecha_actualizacion = schema.fecha_actualizacion
    # ... (no sobrescribimos el estado_proceso)
else:
    # Nueva → insertar con estado inicial
    estado_inicial = (
        "PDF_PENDIENTE" if schema.url_pdf_directo else "NUEVA"
    )
    session.add(Licitacion(estado_proceso=estado_inicial, ...))

```

5.10 utils.py — Funciones auxiliares puras

Este módulo contiene funciones que transforman datos sin efectos secundarios: no hacen HTTP, no tocan la BD, no escriben en disco. Son funciones puras que dado un input siempre devuelven el mismo output. Esto las hace fáciles de testear.

```

# Funciones principales:

# limpiar_texto(): elimina HTML, caracteres de control y espacios múltiples
limpiar_texto("<b>Ayuntamiento de Madrid</b>") → "Ayuntamiento de Madrid"
limpiar_texto(" texto ") → "texto"
limpiar_texto("") → None

# parsear_fecha(): convierte strings de fecha en datetime Python
# El feed usa varios formatos, esta función los maneja todos:
parsear_fecha("2026-04-22T10:30:00Z") → datetime(2026, 4, 22, 10, 30)
parsear_fecha("22/04/2026") → datetime(2026, 4, 22, 0, 0)

# parsear_budget_amount(): extrae el importe del campo UBL multilínea
# El campo devuelve 3 líneas: base sin IVA, con IVA, repetición del base
raw = "154163.5\n186537.83\n154163.5"
parsear_budget_amount(raw) → 154163.5 (tomamos solo la primera línea)

# parsear_summary_pcsp(): extrae campos del texto plano del summary
# Formato: "Id licitación: 4/2025; Órgano: Ayuntamiento X; Estado: ADJ"
parsear_summary_pcsp(summary) → {"expediente": "4/2025", "organo": "..."}

```

6. La Base de Datos

El proyecto usa SQLite como base de datos durante el desarrollo. SQLite es una base de datos que se guarda en un único fichero (sabueso.db) y no requiere instalar ningún servidor. Para producción B2B se migraría a PostgreSQL.

Tablas del sistema

licitaciones: Tabla principal. Una fila por licitación ingestada del feed. Contiene todos los metadatos (título, órgano, presupuesto, CPV, fechas) y el estado del pipeline.

clientes: Una fila por empresa suscrita al servicio. Tiene nombre, email y flag activo/inactivo.

perfiles_interes: Los criterios de búsqueda de cada cliente: palabras clave, prefijos CPV, rango de presupuesto, provincias y descripción para la IA.

matches: El resultado del análisis. Una fila por (licitación, perfil) analizado. Guarda el score de Gemini, la razón y si ya se notificó al cliente.

log_ejecuciones: Auditoría de cada vez que se ejecuta el pipeline. Registra cuántas licitaciones se procesaron, cuántas fueron nuevas y cuántos errores hubo.

Alembic: migraciones de base de datos

Alembic es la herramienta que gestiona los cambios en el esquema de la base de datos. Cuando añadimos una columna nueva o una tabla nueva, no la creamos directamente — creamos una 'migración' que describe el cambio. Alembic aplica las migraciones en orden y mantiene un historial de versiones del esquema.

```
# Comandos Alembic más usados:

# Ver el estado actual (qué migraciones están aplicadas):
alembic current

# Aplicar todas las migraciones pendientes:
alembic upgrade head

# Crear una nueva migración (cuando cambias models.py):
alembic revision --autogenerate -m "descripcion del cambio"

# Ver el historial de migraciones:
alembic history
```

7. Configuración

El proyecto usa variables de entorno para guardar información sensible (como claves de API) fuera del código. Las variables se definen en un archivo .env en la raíz del proyecto que NO se sube a GitHub.

Variables de entorno disponibles

Variable	Valor por defecto	Descripción
GEMINI_API_KEY	(obligatoria)	Clave de API de Google AI Studio
GEMINI_MODEL	gemini-2.5-flash-lite-preview-06-17	Modelo de Gemini a usar
DATABASE_URL	sqlite:///database/sabueso.db	URL de conexión a la BD
MAX_LICITACIONES_POR_RUN	20	Máximo de licitaciones por ejecución
REQUEST_TIMEOUT	30	Timeout de peticiones HTTP en segundos
REQUEST_DELAY_SECONDS	1.0	Pausa entre peticiones (respeto al servidor)
SCORE_RELEVANCIA_MINIMO	60	Score mínimo para considerar RELEVANTE
TEXTO_MINIMO_CARACTERES	100	Chars mínimos para considerar PDF digital
LOG_LEVEL	INFO	Nivel de logs: DEBUG, INFO, WARNING, ERROR

8. Dependencias

Estas son las librerías externas que usa el proyecto y por qué se eligió cada una.

feedparser: Parsea el feed ATOM de la PCSP. Muy tolerante con XML malformado.

beautifulsoup4 + lxml: Estrategias de respaldo para parsear el feed cuando feedparser falla. lxml repara XML, BS4 extrae entradas como último recurso.

requests + tenacity: requests hace las peticiones HTTP. tenacity añade reintentos automáticos con backoff exponencial.

SQLAlchemy: ORM que mapea clases Python a tablas SQL. Permite trabajar con la BD sin escribir SQL directamente.

alembic: Gestiona las migraciones del esquema de la BD. Mantiene un historial de cambios.

python-dotenv: Carga las variables del archivo .env como variables de entorno.

PyMuPDF (fitz): Extrae texto nativo de PDFs digitales. Muy rápido y preciso.

google-genai: Cliente oficial de Google para la API de Gemini. Usado para OCR y análisis de relevancia.

rich: Formatea la salida de la consola con tablas, colores y estilos. Las tablas bonitas que ves en el --stats.

python-dateutil: Parsea fechas en múltiples formatos. Fallback cuando los formatos conocidos no funcionan.

schedule: Permite programar ejecuciones periódicas del pipeline (cada X horas).

pytest + pytest-cov: Framework de tests unitarios y medición de cobertura de código.

9. Cómo Ejecutar el Proyecto

Instalación desde cero

```
# 1. Clonar el repositorio
git clone https://github.com/tu-usuario/sabueso_licitaciones.git
cd sabueso_licitaciones

# 2. Crear entorno virtual con Python 3.12
py -3.12 -m venv .venv

# 3. Activar el entorno virtual (Windows PowerShell)
.venv\Scripts\activate

# 4. Instalar dependencias
pip install -r requirements.txt

# 5. Instalar el proyecto en modo editable
# (permite importar src.* desde cualquier carpeta)
pip install -e .

# 6. Copiar y configurar variables de entorno
copy .env.example .env
# Editar .env y añadir tu GEMINI_API_KEY

# 7. Crear la base de datos (aplicar migraciones)
alembic upgrade head

# 8. Crear un cliente de prueba
python scripts/crear_cliente_prueba.py
```

Comandos CLI disponibles

Comando	Descripción
python main.py	Ejecuta el pipeline completo (Steps 1-4) con 20 licitaciones
python main.py --limite 5	Pipeline completo con máximo 5 licitaciones
python main.py --ping	Verifica conectividad con la PCSP. No toca la BD.
python main.py --stats	Muestra estadísticas de la BD y las 10 más recientes
python main.py --dry-run	Simula la ingesta sin guardar nada en la BD
python main.py --solo-analisis	Solo ejecuta el Step 4 con licitaciones pendientes
python main.py --solo-extraccion	Solo ejecuta el Step 3 con PDFs descargados

10. Estado Actual y Roadmap

Estado verificado el 17 de junio de 2026

Step	Módulo	Estado	Notas
Step 1	scraper_atom.py	■ Funciona	350 licitaciones en el feed actual
Step 2	downloader.py	■ Funciona	Descarga directa sin Playwright
Step 3	extractor.py	■ Funciona	PyMuPDF OK. OCR con Gemini OK.
Step 4	analizador.py + ia_client.py	■ Funciona	Gemini 2.5 Flash-Lite activo
Step 5	(no implementado)	■ Pendiente	Canal: WhatsApp o email

Próximos pasos (Roadmap)

Step 5 — Notificaciones: Implementar el módulo de notificaciones. Canal prioritario: email (simple, sin dependencias de terceros) o WhatsApp Business API. Cuando una licitación es RELEVANTE, enviar un resumen al email del cliente con el título, órgano, presupuesto, score y razón de la IA.

Panel web de administración: Una interfaz web sencilla donde el cliente pueda ver sus licitaciones relevantes, gestionar sus perfiles y configurar sus criterios. Stack sugerido: FastAPI + React o Flask + HTML simple.

Scheduler automático: El pipeline ya tiene la librería schedule instalada. Configurar ejecución automática cada 24 horas para que el sistema funcione sin intervención manual.

Migración a PostgreSQL: Para producción con varios clientes simultáneos, migrar de SQLite a PostgreSQL. Alembic hace esta migración sencilla — solo hay que cambiar la DATABASE_URL.

Mejora del sistema de errores: Las 6 licitaciones en estado ERROR de la BD antigua quedaron bloqueadas. Añadir un comando --reintentar-errores que las reponga en el estado anterior para que vuelvan a procesarse.